Práctica de Clase: Desarrollo de App "Limonada" con Android Studio y Kotlin Compose

---

Esta guía interactiva y detallada está diseñada para guiar a los estudiantes paso a paso en el desarrollo de una aplicación móvil interactiva utilizando **Jetpack Compose** y **Kotlin**. A través de este proyecto, los alumnos aprenderán conceptos fundamentales de estado, diseño de interfaz reactiva y buenas prácticas en Android.

1. Objetivos de Aprendizaje

- **Gestión de Estado (State Management):** Comprender y aplicar el uso de remember, mutableStateOf y delegados de propiedades (by) para refrescar la interfaz automáticamente.
- **Flujo Dinámico y Control de Navegación:** Implementar estructuras condicionales (when) para alternar vistas e imágenes en respuesta a la interacción del usuario.
- **Reusabilidad de Componentes Composables:** Crear componentes modulares y desacoplados con parámetros y funciones de retorno de llamada (*lambda callbacks*).
- **Estructuras de Diseño de Material Design 3:** Emplear contenedores y componentes estándar como Scaffold, CenterAlignedTopAppBar, Surface, Column, Box y Button.
- **Gestión de Recursos de Android:** Organizar y referenciar adecuadamente recursos numéricos, textos e imágenes mediante dimensionResource, stringResource y painterResource.

2. Estructura de Recursos del Proyecto

2.1 Cadenas de Texto (res/values/strings.xml)

Incorpore las siguientes definiciones en el archivo strings.xml para garantizar la internacionalización y soporte de accesibilidad:

```
<resources>
    <string name="app_name">Limonada</string>
    <string name="lemon_select">Haz clic para seleccionar un limón</string>
    <string name="lemon_squeeze">Haz clic para exprimir el limón</string>
```

```
    <string name="lemon_drink">Haz clic para beber la limonada</string>
    <string name="lemon_empty_glass">Haz clic para empezar de
nuevo</string>
    <string name="lemon_tree_content_description">Limonero</string>
    <string name="lemon_content_description">Limón</string>
    <string name="lemonade_content_description">Vaso de limonada</string>
    <string name="empty_glass_content_description">Vaso vacío</string>
</resources>
```

2.2 Dimensiones y Espaciados (res/values/dimens.xml)

Cree el archivo dimens.xml dentro de la carpeta res/values/ para estandarizar bordes, tamaños de imagen y rellenos:

```
<resources>
    <dimen name="button_corner_radius">40dp</dimen>
    <dimen name="button_image_width">128dp</dimen>
    <dimen name="button_image_height">160dp</dimen>
    <dimen name="button_interior_padding">16dp</dimen>
    <dimen name="padding_vertical">16dp</dimen>
</resources>
```

2.3 Recursos Gráficos (res/drawable/)

Asegúrese de importar las siguientes 4 imágenes en formato PNG o Vector a la carpeta res/drawable:

Nombre del Archivo	Descripción del Recurso	Fase de la Aplicación
lemon_tree.png	Imagen de un árbol de limones	Paso 1: Selección del limón
lemon_squeeze.png	Imagen de un limón listo para exprimir	Paso 2: Exprimir el limón
lemon_drink.png	Imagen de un vaso con limonada fresca	Paso 3: Beber la limonada
lemon_restart.png	Imagen de un vaso vacío	Paso 4: Reiniciar proceso

3. Paso a Paso del Código Fuente

Paso 1: Construcción del Componente Reutilizable (LemonTextAndImage)

Diseñamos una función composable que encapsula el botón con la imagen redondeada y el texto explicativo. Esto evita la duplicación de código en los 4 estados de la app.

```
@Composable
fun LemonTextAndImage(
    textLabelResourceId: Int,
    drawableResourceId: Int,
    contentDescriptionResourceId: Int,
    onImageClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Box(
        modifier = modifier
    ) {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            Button(
                onClick = onImageClick,
                shape =
RoundedCornerShape(dimensionResource(R.dimen.button_corner_radius)),
                colors = ButtonDefaults.buttonColors(containerColor =
MaterialTheme.colorScheme.tertiaryContainer)
            ) {
                Image(
                    painter = painterResource(drawableResourceId),
                    contentDescription =
stringResource(contentDescriptionResourceId),
                    modifier = Modifier

.width(dimensionResource(R.dimen.button_image_width))

.height(dimensionResource(R.dimen.button_image_height))

.padding(dimensionResource(R.dimen.button_interior_padding))
                )
            }
```

```
            Spacer(modifier =
Modifier.height(dimensionResource(R.dimen.padding_vertical)))
            Text(
                text = stringResource(textLabelResourceId),
                style = MaterialTheme.typography.bodyLarge
            )
        }
    }
}
```

Paso 2: Creación de la Estructura Principal y Barra Superior (Scaffold)

Utilizamos el componente Scaffold para proveer una estructura genérica de Material Design que incluye una barra superior de título centrada (CenterAlignedTopAppBar).

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LemonadeApp() {
    var currentStep by remember { mutableStateOf(1) }
    var squeezeCount by remember { mutableStateOf(0) }

    Scaffold(
        topBar = {
            CenterAlignedTopAppBar(
                title = {
                    Text(
                        text = "Limonada",
                        fontWeight = FontWeight.Bold
                    )
                },
                colors = TopAppBarDefaults.largeTopAppBarColors(
                    containerColor =
MaterialTheme.colorScheme.primaryContainer
                )
            )
        }
    ) { innerPadding ->
        Surface(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .background(MaterialTheme.colorScheme.tertiaryContainer),
            color = MaterialTheme.colorScheme.background
```

```
        ) {
            // Control de flujo (Paso 3)
        }
    }
}
```

Paso 3: Lógica del Estado y Flujo Interactivo

Ubicado dentro del bloque Surface, la sentencia when se encarga de conmutar las vistas y actualizar el número de clics requeridos para exprimir el limón:

```
when (currentStep) {
    1 -> {
        LemonTextAndImage(
            textLabelResourceId = R.string.lemon_select,
            drawableResourceId = R.drawable.lemon_tree,
            contentDescriptionResourceId =
R.string.lemon_tree_content_description,
            onImageClick = {
                currentStep = 2
                squeezeCount = (2..4).random()
            }
        )
    }
    2 -> {
        LemonTextAndImage(
            textLabelResourceId = R.string.lemon_squeeze,
            drawableResourceId = R.drawable.lemon_squeeze,
            contentDescriptionResourceId =
R.string.lemon_content_description,
            onImageClick = {
                squeezeCount--
                if (squeezeCount == 0) {
                    currentStep = 3
                }
            }
        )
    }
    3 -> {
        LemonTextAndImage(
            textLabelResourceId = R.string.lemon_drink,
            drawableResourceId = R.drawable.lemon_drink,
            contentDescriptionResourceId =
```

```
R.string.lemonade_content_description,
            onImageClick = {
                currentStep = 4
            }
        )
    }
    4 -> {
        LemonTextAndImage(
            textLabelResourceId = R.string.lemon_empty_glass,
            drawableResourceId = R.drawable.lemon_restart,
            contentDescriptionResourceId =
R.string.empty_glass_content_description,
            onImageClick = {
                currentStep = 1
            }
        )
    }
}
```

Paso 4: Punto de Entrada y Vista Previa (MainActivity)

Finalmente, configuramos la actividad principal e incluimos la anotación @Preview para visualizar el diseño en tiempo de desarrollo.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            LemonadeTheme {
                LemonadeApp()
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun DefaultPreview() {
    LemonadeTheme {
        LemonadeApp()
    }
```

```
}
```

4. Retos Extracurriculares para Alumnos Avanzados

Desafíos opcionales para profundizar:

1. **Indicador de Toques Restantes:** Añadir un componente Text debajo de la imagen durante el paso 2 que muestre cuántos toques faltan para terminar de exprimir el limón (ejemplo: *"Toques restantes: $squeezeCount"*).
2. **Persistencia en Rotación:** Sustituir remember { mutableStateOf(...) } por rememberSaveable { mutableStateOf(...) } y probar qué ocurre con el contador al girar el dispositivo.
3. **Animación de Transición:** Agregar un efecto suave de cambio de tamaño o de opacidad al pasar entre cada uno de los estados de la limonada.